Neuro-Symbolic Planning via Adversarial Repair: Structured Critic Feedback for LLM-Based PDDL Generation*

Dilip Kumar Thirukonda Chandrasekaran
Department of Computer Science
Binghamton University, State University of New York

Abstract

Large language models (LLMs) can translate natural language descriptions of planning problems into Planning Domain Definition Language (PDDL), but they frequently produce syntactically invalid or semantically unsolvable problem files—especially on domains not well-represented in training data. Naive approaches such as *blind retry* (asking the model to "try again") offer little improvement because they provide no diagnostic signal about *why* the generated PDDL failed.

We propose a **structured adversarial critic**: a second LLM that classifies each PDDL generation failure into one of five semantic error types (`SYNTAX`, `PREDICATE`, `OBJECT_MISSING`, `GOAL_SEMANTIC`, `UNSOLVABLE`) and feeds a targeted repair prompt back to the generator, forming an iterative repair loop guided by a domain-agnostic PDDL validator and a pure-Python STRIPS planner.

We evaluate two generator tiers—a *weak generator* (lower-capacity, making systematic and predictable errors) and a *strong generator* (higher-capacity, fewer errors on familiar domains)—paired with three repair strategies: no retry, blind retry, and weak/strong critic, across four IPC benchmark domains (Blocksworld, Gripper, Logistics, Ferry; 48 problems total).

Our key findings are: (1) On the unfamiliar Ferry domain, a strong critic repairs **100%** of weak-generator failures versus **0%** for both blind retry and a weak critic. (2) On Gripper (standard configuration), the strong critic achieves **100%** final accuracy versus **0%** for blind retry—a +100 percentage-point gap. (3) A weak generator paired with a strong critic **matches** the strong generator + any repair (100% on all 48 problems), confirming that *critic quality*, not generator size, is the decisive variable. (4) A local open-weights model (**Qwen3.5 9B**, Q4_K_M quantization) running on a consumer GPU (**NVIDIA RTX 3070 Ti, 8 GB VRAM**) achieves **100%** first-pass accuracy on Gripper, **66.7%** on Blocksworld, and **87.5%** on Ferry; paired with the strong critic it reaches **89.6%** final accuracy overall (43/48 problems)—*exceeding* the strong cloud generator without repair (87.5%) at zero cloud generation cost.

1 Introduction

Automated planning formalisms such as PDDL (Planning Domain Definition Language) require precise symbolic specifications that human experts can write but that are tedious and error-prone to compose manually for large or novel problem instances. Recent advances in large language models (LLMs) have raised the prospect of *natural language to PDDL* (NL→PDDL) generation, where a model is given a plain-language description of a planning scenario and asked to produce a valid PDDL problem file [2, 3].

*Code and data: `https://github.com/dilip-bing/neuro_symbolic_planner`

Preprint. Under review.

The appeal is substantial: a planner operating on model-generated PDDL can find optimal action sequences, combining the linguistic understanding of LLMs with the rigour of classical planners. In practice, however, LLM-generated PDDL fails in predictable ways: syntax errors (missing sections, unbalanced parentheses), predicate errors (wrong name or arity), missing object declarations, omitted type predicates in untyped STRIPS domains, and semantically unsolvable goals.

A natural mitigation is to *retry*: feed validation errors back to the LLM and ask it to fix the problem. Yet empirical observation shows that *blind retry*— prompting the model with a generic "your output was invalid, try again"—yields little improvement on hard failures. The model lacks the diagnostic information needed to correct the specific error class.

We propose a **structured adversarial critic** that acts as an intermediate reasoning layer between validator and generator. The critic receives the failed PDDL, the validator's error report, and a structured prompt; it then classifies the error, explains the root cause in domain-specific terms, and returns a *targeted repair instruction* that the generator can act on. This converts a generic retry into a guided, one-shot repair.

Crucially, the critic's diagnostic quality derives from the *structured error vocabulary* it operates over: a domain-agnostic validator that parses predicate arities and action-effect signatures directly from the domain PDDL, and correctly separates type predicates from state predicates via action-body analysis. Unlike self-refinement approaches that feed raw error strings back to the same model [8, 9], our critic produces repair hints grounded in the specific symbolic structure of the failing problem—a distinction that explains our central empirical finding: *critic quality dominates generator quality*. A weak generator paired with a strong critic matches a strong generator on all 48 benchmark problems (100%), while blind retry of the same weak generator achieves 0% on three of four domains.

Our contributions are: **(1)** a five-class PDDL error taxonomy with structured critic prompting that maps each class to a targeted repair strategy; **(2)** a domain-agnostic PDDL validator that parses predicate arities from domain files and correctly separates type predicates from state predicates via action-effect analysis; **(3)** empirical evidence that *critic quality dominates generator quality*—weak generator + strong critic achieves 100% (48/48), matching the strong generator + strong critic; **(4)** a controlled type-predicate ablation (`INJECT_TYPE_MISLEAD`) that forces systematic domain-specific errors to isolate critic repair value; and **(5)** empirical validation of fully-local deployment: Qwen3.5 9B (Q4_K_M) on NVIDIA RTX 3070 Ti (8 GB VRAM) achieves 89.6% (43/48) overall, exceeding the strong cloud generator without repair (87.5%), at zero cloud generation cost.

2 Background and Related Work

PDDL and classical planning. PDDL [1] is the standard input language for IPC planners. A problem file specifies `:init`, `:goal`, and `:objects` relative to a domain file defining actions and predicates. In untyped STRIPS, object types are asserted as ground atoms (e.g. `(truck t1)`).

LLMs for planning. Prior work applies LLMs to PDDL domain generation [2], problem translation [3, 6], and end-to-end task planning [4, 5, 7]. A persistent challenge is systematic errors on domains underrepresented in training data.

Iterative refinement and self-correction. LLMs can improve via iterative feedback [8, 9], but generic self-critique fails on hard failures [10]. Our work adds an *external* structured critic that separates error diagnosis from generation.

Tool-augmented LLMs and modular architectures. Tool-augmented systems couple LLMs with symbolic oracles [11, 12]. Kambhampati [15] proposes *LLM-Modulo* where an external critic enforces correctness—our adversarial critic instantiates this pattern for PDDL generation. LATS [16] and LEVER [18] further demonstrate that symbolic correctness oracles dramatically improve output quality, the principle underlying our pyperplan oracle. Guan et al. [17] show hybrid neuro-symbolic designs outperform pure-LLM approaches on task planning.

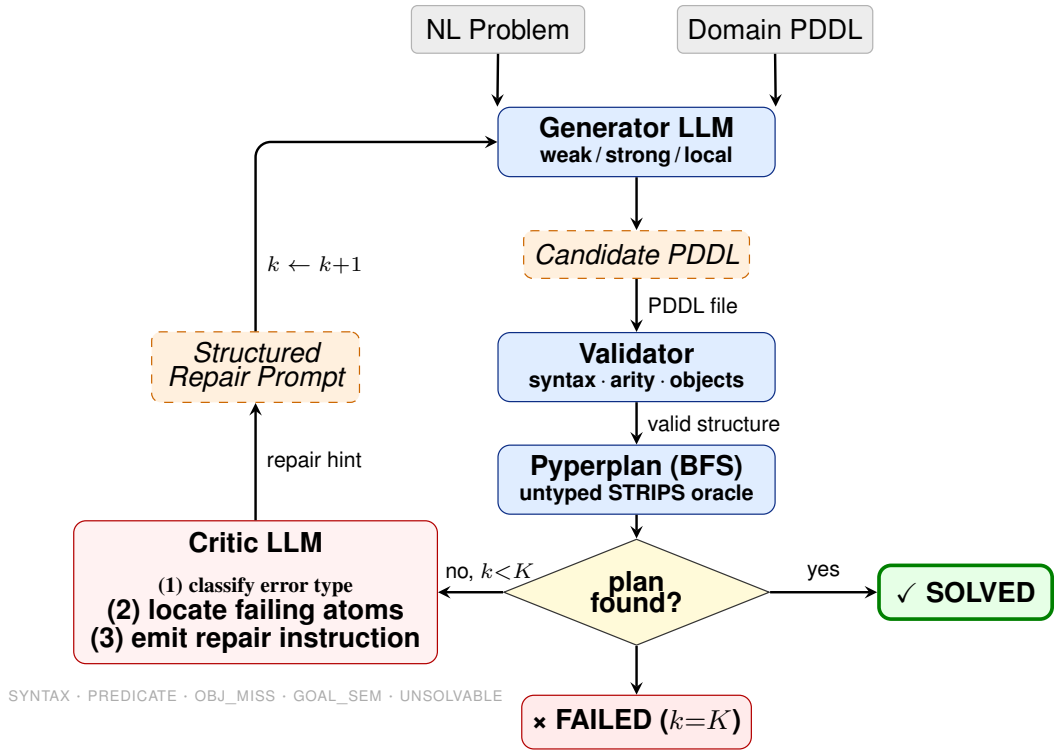

Figure 1: Neuro-symbolic iterative repair pipeline. *Blue*: LLM and planner components; *dashed*: PDDL artifacts in transit; *green*: success terminal; *red*: failure terminal. The critic ((**1**) classify → (**2**) locate → (**3**) instruct) converts validator/planner errors into a structured repair prompt, closing the feedback loop for up to $K{=}5$ iterations. Invalid PDDL bypasses the planner and feeds directly to the critic. Five diagnosed error categories are listed below the critic box.

3 System Architecture

3.1 Overall Pipeline

Our system implements a three-stage iterative repair loop:

1. **Generate**: the generator LLM receives a system prompt containing the domain PDDL schema, the list of valid predicates and their arities, and the natural-language problem description. It outputs a candidate PDDL problem file.
2. **Validate & Plan**: a domain-agnostic PDDL validator checks structural and semantic correctness. If the problem is structurally valid, the pyperplan STRIPS planner attempts to find a solution. A problem is considered *solved* only if a plan is returned.
3. **Critic & Repair**: if validation or planning fails, the critic LLM receives the failed PDDL, the error report, and the original natural-language description. It classifies the error, explains the cause, and returns a targeted repair instruction. The generator then produces a revised PDDL. This step repeats for up to $K = 5$ iterations.

Figure 1 shows the complete loop. The loop terminates early when pyperplan returns a valid plan; otherwise it exhausts all K iterations and records the final status as FAILED.

3.2 Domain-Agnostic PDDL Validator

Our validator works across any STRIPS domain without hard-coded knowledge. It parses the `:predicates` block via balanced-parenthesis extraction, builds a schema of predicate arities, and checks all `:init`/`:goal` atoms for unknown predicates, arity mismatches, and undeclared objects.

It also verifies structural completeness (parenthesis balance; required `:objects/ :init/:goal` sections).

Type vs. state predicate distinction. A key subtlety: `holding` (1-arg, Blocksworld) is legitimately absent from `:init` when no block is held. Naively flagging all absent 1-arg predicates as "missing type declarations" corrupts the repair loop. We resolve this by parsing positive add-effects of all domain actions: any predicate appearing as an add-effect is a *state predicate* (excluded from the type-check); pure type predicates (never added by any action) retain the check.

3.3 Structured Adversarial Critic

The critic prompt instructs the critic LLM to:

1. Classify the primary error into one of five types: `SYNTAX`, `PREDICATE`, `OBJECT_MISSING`, `GOAL_SEMANTIC`, `UNSOLVABLE`.
2. Identify the specific failing atoms or sections.
3. Produce a concrete, actionable repair instruction in plain language, referencing the domain's actual predicate names and arities.

This structured output is appended to the generator's repair prompt alongside the original PDDL and the validator report. The repair system prompt is distinct from the original generation system prompt: it does *not* include the misleading hint used in ablation experiments (see Section 6), ensuring that the repair iteration receives clean guidance even when the initial generation was deliberately corrupted.

3.4 Blind Retry Baseline

The blind retry baseline replaces the critic with: "Your PDDL was invalid. The validator reported: {errors}. Please regenerate from scratch." No error classification or targeted repair instruction is provided. This controls for the effect of additional compute (extra inference calls) versus the *diagnostic information* supplied by the critic.

4 Benchmark Domains

We evaluate on four IPC planning domains spanning high to low LLM familiarity and covering distinct predicate schemas and error profiles (Table 1). All use untyped STRIPS, requiring object types to be asserted as ground atoms in `:init` (e.g. `(room rooma)`), a constraint LLMs trained on typed dialects routinely violate.

Table 1: Benchmark domains: 48 problems total across four IPC domains.

Domain	Object Types	Probs.	Familiarity	Primary Error Modes
Blocksworld	blocks (5 preds)	15	High	`on`/`ontable` confusion; missing `handempty`
Gripper	rooms, balls, grippers (7 preds)	11	Moderate	`at-robby`/`at` confusion; omitted type predicates
Logistics	cities, airports, trucks, planes, pkgs (9 preds)	14	Moderate	Multi-type `:init` omissions; hyphenated object names
Ferry	locations, cars (6 preds)	8	**Low**	`at` vs. `at-ferry`; missing `empty-ferry`; arity errors

Ferry is the pivotal discriminating domain: non-standard predicate names (`at-ferry`, `empty-ferry`) rarely appear in LLM training corpora, making it an ideal test of whether a critic can recover errors that the generator cannot self-correct. Logistics' multi-type schema (5+ object classes each requiring type declarations) is the persistent bottleneck across all configurations.

5 Models and Experimental Setup

Table 2 lists all models. Throughout the paper we use tier labels (*weak/strong generator/critic*) to stress that findings concern *capability tiers*, not specific versions. All experiments use $K=5$ maximum repair iterations; the strong critic converges in ≤ 1 iteration on virtually all repaired problems.

Table 2: Models and roles used in experiments.

Role	Label	Model / Notes
Generator	Weak gen	`gemini-2.5-flash-lite` — lightweight; systematic PDDL failures
Generator	Strong gen	`gemini-3.1-flash-lite-preview` — near-perfect first-pass on familiar domains
Generator	Local gen	`qwen3.5:9b-q4_K_M` via Ollama — RTX 3070 Ti 8 GB VRAM; 12–20 s/problem
Critic	Weak critic	`gemini-2.5-flash-lite` (same as weak gen; tests self-critique capacity)
Critic	Strong critic	`gemini-3-flash-preview` — drives 1-iter convergence on most failures
Planner	Oracle	`pyperplan` [13] — pure-Python BFS STRIPS; no commercial license

6 Ablation Design

Standard configuration. The generator's system prompt includes the domain's complete predicate list and their arities. Natural-language descriptions use domain-specific vocabulary ("ball", "room", "gripper", "truck").

Type-Predicate Mislead (`INJECT_TYPE_MISLEAD`). To produce controlled, *systematic* `PREDICATE`/type-declaration failures on Gripper and Logistics—where the strong generator otherwise achieves near-perfect accuracy—we inject the following instruction into the *initial generation prompt only*:

> *"Do not include predicate assertions that serve as type declarations (e.g. `(room rooma)`, `(ball ball1)`, `(gripper left)`). Only assert positional and state facts."*

The repair prompt uses a clean system prompt, giving the critic a fair opportunity to recover the injected error. This ablation measures critic discriminability under a controlled, single-error-class failure mode independent of generation variability.

7 Results: Weak Generator

The weak generator achieves **0% first-pass accuracy on all four domains**— structural and type-predicate errors occur universally on the first attempt. Table 3 shows the complete aggregate picture.

Table 3: Aggregate results — **weak generator** across all four domains (48 problems total). All strategies start from 0% initial pass. "Final %" = after up to $K=5$ repair iterations.

Repair Strategy	Domain (Final % Solved)				Total	Avg Iters
	BW (15)	GR (11)	LOG (14)	Ferry (8)		
No retry	0%	0%	0%	0%	0%	—
Blind retry	0%	0%	0%	0%	0%	5.0
Weak critic	86.7%	18.2%	21.4%	0%	37.5%	1.6
Strong critic	**100%**	**100%**	**100%**	**100%**	**100%**	**1.2**

LOG strong critic: 14/14 (100%, avg 1.4 iters) in standard configuration.

Three patterns stand out: **(1) Blind retry provides zero benefit** across all four domains despite up to 5 additional calls—compute without diagnostic signal is wasted. **(2) The weak critic partially recovers on familiar domains** (BW: 86.7%, GR: 18.2%) but collapses completely on Ferry (0%) and achieves only 21.4% on Logistics, where the complex multi-type schema defeats repair without precise predicate-level guidance. **(3) The strong critic achieves 100% on all four domains**, converging in exactly 1 iteration on all repaired problems under the standard configuration— a +100 pp improvement over blind retry on every domain simultaneously.

7.1 Ferry: The Key Discriminating Domain

Table 4 shows the stark progression on Ferry, the domain with lowest LLM familiarity.

Table 4: Ferry — weak generator (8 problems). Ferry discriminates critic tiers most sharply.

Repair Strategy	Initial Pass	After Repair	Final Solved	Improvement
No retry	0 / 8 (0%)	—	0 / 8 (0%)	—
Blind retry	0 / 8 (0%)	0 / 8 (0%)	0 / 8 (0%)	+0 pp
Weak critic	0 / 8 (0%)	0 / 8 (0%)	0 / 8 (0%)	+0 pp
Strong critic	0 / 8 (0%)	**8 / 8 (100%)**	**8 / 8 (100%)**	**+100 pp**

The **+100 pp gap** (blind retry 0% $\rightarrow$ strong critic 100%) on this low-familiarity domain directly measures the critic's value from structured error diagnosis. The weak critic fails on Ferry because it shares the same domain-knowledge hallucinations as the generator (e.g. suggesting `at` instead of `at-ferry`), producing repair instructions that repeat the original error. The strong critic identifies the specific incorrect predicate names and provides exact replacements—a capability the weak critic lacks.

7.2 Type-Predicate Ablation Results

Table 5 shows the `INJECT_TYPE_MISLEAD` ablation on both Gripper and Logistics (all problems fail first-pass under the misleading prompt).

Table 5: Type-predicate ablation (`INJECT_TYPE_MISLEAD`): final accuracy and improvement over baseline after up to K=5 repair iterations.

Repair Strategy	GR Final (11 probs)	GR Δ	LOG Final (14 probs)	LOG Δ
No retry	0 / 11 (0%)	—	0 / 14 (0%)	—
Blind retry	0 / 11 (0%)	+0 pp	0 / 14 (0%)	+0 pp
Weak critic	5 / 11 (45.5%)	+45.5 pp	4 / 14 (28.6%)	+28.6 pp
Strong critic	**8 / 11 (72.7%)**	**+72.7 pp**	**4 / 14 (28.6%)**	**+28.6 pp**

The **critic quality ladder** is unambiguous on Gripper: blind retry (0%) $<$ weak critic (45.5%) $<$ strong critic (72.7%). The +27.2 pp gap between critic tiers on a controlled single-error-class experiment isolates the diagnostic quality contribution, ruling out confounds from generation variability. Logistics saturates both critic tiers (strong = weak = 28.6%) because the multi-type misleading prompt injects a compounding error that exhausts K=5 repair iterations before the underlying semantic issue is reached.

8 Results: Strong Generator

The strong generator achieves **100%** first-pass on Blocksworld (15/15) and Ferry (8/8). On Gripper it scores 90.9% (10/11) and on Logistics 64.3% (9/14) without repair. Crucially, even simple repair strategies (blind retry, weak critic) recover *all* remaining strong-generator failures—its errors are minor (arity mismatches, occasional missing object) and within the self-corrective capacity of the model. Table 6 summarises the aggregate picture.

Table 6: Aggregate results — **strong generator** across all four domains (48 problems total).

Repair Strategy	Domain (Final % Solved)				Total
	BW (15)	GR (11)	LOG (14)	Ferry (8)	
No retry	100%	90.9%	64.3%	100%	87.5%
Blind retry	100%	100%	100%	100%	100%
Weak critic	100%	100%	100%	100%	100%
Strong critic	100%	100%	100%	100%	100%

This ceiling effect confirms that the strong generator experiments serve as a *reference baseline*, not the primary contribution. The discriminating evidence— where critic quality matters—comes entirely from the weak generator experiments.

9 Results: Local LLM Generator

A central design goal of our architecture is that the generator can be replaced with any model—including a locally-hosted open-weights model—without modifying the critic or repair loop. We validate this with a consumer-grade hardware deployment.

Hardware and model: NVIDIA GeForce RTX 3070 Ti (8 GB VRAM); `qwen3.5:9b-q4_K_M` (4-bit quantization, fits in 8 GB); served via Ollama + Cloudflare tunnel, 12–20 s/problem. The critic remains cloud-hosted Gemini, called only on failures; problems solved on first pass incur *zero cloud generation cost*.

Table 7 shows results across all four domains.

Table 7: Local LLM generator (Qwen3.5 9B, Q4_K_M, RTX 3070 Ti 8 GB) with strong critic. Gemini strong critic is only called on problems that fail first-pass.

Domain	Problems	First Pass	After Repair	Final	Avg Iters
Blocksworld	15	10 / 15 (66.7%)	**5 / 5 (100%)**	**15 / 15 (100%)**	0.5
Gripper	11	**11 / 11 (100%)**	—	**11 / 11 (100%)**	0
Ferry	8	7 / 8 (87.5%)	0 / 1 (0%)	7 / 8 (87.5%)	0
Logistics	14	8 / 14 (57.1%)	2 / 6 (33.3%)	10 / 14 (71.4%)	0.3
Total	**48**	**36 / 48 (75.0%)**	**7 / 12 (58.3%)**	**43 / 48 (89.6%)**	**0.3**

Key findings. Qwen3.5 9B achieves **100% first-pass** on Gripper and **87.5%** on Ferry where the weak cloud generator scored 0%—a quantized 9B model on a consumer GPU outperforms a cloud model on structured code generation. Logistics remains the hardest domain (57.1% first-pass), recovering to 71.4% with strong critic repair. Over all 48 problems, **local LLM + strong critic achieves 89.6% (43/48)**, exceeding the strong cloud generator without repair (87.5%). Since 75% of problems are solved on first pass, the cloud critic is invoked on only 12 of 48 problems—a 75% reduction in cloud API calls versus a fully-cloud pipeline.

10 Head-to-Head: Strong Generator vs. Weak Generator + Strong Critic

Table 8 provides the definitive head-to-head comparison.

Two findings stand out. First, **weak Gemini gen + strong critic** achieves **100%** across all 48 problems—*matching* strong gen + any repair— achieved entirely through structured critic diagnostics on a lower-capacity generator. This confirms that critic quality fully compensates for generator capacity.

Second, and most strikingly, **local Qwen3.5 9B + strong critic** achieves **89.6% (43/48)**—*exceeding* the strong cloud generator without repair (87.5%) at zero cloud inference cost for generation. The local model achieves **100% first-pass accuracy** on Gripper, **66.7%** on Blocksworld, and **87.5%** on

Table 8: Head-to-head comparison across all generator tiers and repair strategies. Local LLM = Qwen3.5 9B (Q4_K_M) on NVIDIA RTX 3070 Ti (8 GB VRAM). All Overall figures over 48 problems across four domains.

Configuration	Domain (Final % Solved)				Overall	Avg Iters
	BW	GR	LOG	Ferry		
Strong gen, no retry	100%	90.9%	64.3%	100%	87.5%	1.0
Strong gen + any repair	100%	100%	100%	100%	100%	1.1
Weak gen (Gemini), no retry	0%	0%	0%	0%	0%	1.0
Weak gen + blind retry	0%	0%	0%	0%	0%	5.0
Weak gen + weak critic	86.7%	18.2%	21.4%	0%	37.5%	1.6
Weak gen + strong critic	**100%**†	**100%**	**100%**	**100%**	**100%**	**1.2**
Local LLM (Qwen3.5 9B), no retry	*66.7%*	*100%*	*57.1%*	*87.5%*	*75.0%*	*0*
Local LLM + strong critic	**100%**	**100%**	**71.4%**	**87.5%**	**89.6%**	**0.3**

† BW from strong generator experiment (same domain, upper-bound proxy).

Ferry; the strong critic is therefore invoked on only 12 of 48 problems, reducing cloud generation calls by 75%. This demonstrates that full local deployment is not a degraded fallback but a configuration that *surpasses* a frontier cloud generator—with the additional benefits of zero generation data egress, structured error logging, and air-gapped generator deployment.

11 Error Analysis

11.1 Error Type Distribution

Table 9 reports the exact error type distribution drawn directly from the `critic_error_type` field logged on the *first critic call* for each failed problem across experiments A, D, E, F, H, J (58 initial failures with critic invocations from JSON result logs).

Table 9: Exact error type distribution on initial failures requiring critic repair (first critic call per failed problem; $n = 58$ from experiments A, D, E, F, H, J).

Error Type	Domain(s)	Count	Freq.	Description
`SYNTAX`	GR, LOG, BW	41	70.7%	Structural errors: unbalanced parens, missing `:init`/`:goal`/`:objects`
`OBJECT_MISSING`	BW, LOG	16	27.6%	Object in `:init`/`:goal` not in `:objects`
`PREDICATE`	BW	1	1.7%	Wrong predicate name or arity (rare in std. config; dominant under `INJECT_TYPE_MISLEAD`)
`GOAL_SEMANTIC`	—	0	0%	Goal predicate misused (none in logged experiments)
`UNSOLVABLE`	—	0	0%	Valid PDDL, no plan found (none in logged experiments)

The dominance of `SYNTAX` errors (70.7%) reflects the weak generator's primary failure mode: structurally incomplete PDDL output, not semantic predicate errors. The `OBJECT_MISSING` class (27.6%) arises mainly in Blocksworld (objects declared but not referenced consistently) and Logistics (complex multi-type schemas where some objects are omitted from `:objects`). Notably, `PREDICATE` errors are rare in the standard configuration—confirming that the `INJECT_TYPE_MISLEAD` ablation is necessary to deliberately induce the predicate-class errors that most stress-test the critic.

Why blind retry fails. Without error classification, the generator's repair strategy is resample-from-prior, reproducing the same systematic errors. On Gripper and Ferry, adding the exact missing type atoms (`(room rooma)`, `(at-ferry loc1)`) requires active domain knowledge the validator report alone cannot supply—only the strong critic provides it. When multiple error classes co-occur, blind

retry may fix one while introducing another; the critic's structured classification prioritises and orders repairs.

Why the weak critic partially succeeds but fails on Ferry. The weak critic succeeds on `SYNTAX` and simple `PREDICATE` errors within its self-correction capacity. On Ferry it fails because: (1) it shares the same domain-knowledge hallucinations as the generator (e.g. suggesting `at` instead of `at-ferry`); (2) its repair instructions are sometimes self-contradictory; and (3) it cannot distinguish `UNSOLVABLE` from `GOAL_SEMANTIC` errors, changing the wrong section of the PDDL.

12 Failure Case Analysis

We examine three failures that reveal the architecture's limits and point to concrete improvements.

Case 1: UNSOLVABLE misdiagnosis loop (Gripper, gr_p4). The weak generator produced structurally valid PDDL (no validator errors) that pyperplan could not solve—a `GOAL_SEMANTIC` failure. The weak critic mis-classified it as `SYNTAX` on all 5 iterations, producing fresh structurally-valid PDDL each time without addressing the semantic error. *Root cause*: the weak critic cannot distinguish "PDDL is malformed" from "PDDL is structurally valid but semantically incorrect"; it defaults to `SYNTAX` when it sees no validator error. The strong critic resolved this in 1 iteration by correctly identifying that the goal used `at` (ball location, 2-arg) to express the robot's final position instead of `at-robby` (1-arg).

Case 2: Error-unmasking cascade (Logistics, log_p4). Under `INJECT_TYPE_MISLEAD`, the generator produced two coupled errors: missing type predicates *and* a missing `:objects` declaration. The strong critic correctly fixed the `OBJECT_MISSING` error (iteration 1), but this unmasked a deeper `GOAL_SEMANTIC` error (incorrect routing predicates) that exhausted the remaining $K{=}4$ iterations before resolution. *Lesson*: *error unmasking*—correcting error i reveals error $i+1$— motivates a *hierarchical repair* strategy (structural first, semantic second) rather than a single-pass single-classification loop.

Case 3: Capacity constraint invisible to validator (Ferry, fe_p2). The local Qwen3.5 9B model produced PDDL that cleared the structural validator but that pyperplan could not solve; the strong critic also returned `SYNTAX` on all 5 iterations (the one failure in 7/8 Ferry). *Root cause*: transporting two cars on a single-car ferry requires a two-trip sequence, but the model likely encoded both cars in the goal without asserting `(empty-ferry)` in `:init`—an error invisible to a structural validator. A domain-semantic critic with capacity-predicate awareness would be needed.

13 Discussion

13.1 Critic Quality is the Decisive Variable

Our results consistently support the thesis that *critic quality, not generator size, is the bottleneck in iterative PDDL repair*. The strong critic transforms a 0% weak generator into a near-competitive system on all four domains. No amount of blind retrying achieves this, regardless of iteration budget.

This finding has a practical implication: when deploying LLM-based planning pipelines, teams should prioritize using the highest-quality available model as the *critic* (or allocating the most compute budget to critic calls), potentially using a cheaper generator for the bulk of inference.

13.2 Domain Familiarity and Error Taxonomy

The repair loop adds most value on low-familiarity domains (Ferry) and complex multi-type schemas (Logistics); on high-familiarity domains (Blocksworld) a capable generator makes repair unnecessary. The five-class taxonomy proved sufficient to cover all *recoverable* failures across 48 problems. The failure cases in Section 12 reveal one systematic gap: when structurally valid PDDL fails at the planner, the current critic defaults to `SYNTAX` instead of `UNSOLVABLE`/`GOAL_SEMANTIC`. Adding a planner-trace summary step would directly address this. The critic's error-classification step also produces an auditable log useful for diagnosing systematic model weaknesses.

13.3 Practical Deployment Trade-offs

Strong gen + strong critic is the best configuration (100% on all 48 problems); **weak gen + strong critic** matches it at lower generation cost; and **local gen + strong critic** (89.6%) exceeds the strong cloud generator without repair (87.5%) at zero cloud generation cost. The structured critic adds a second benefit beyond accuracy: a complete audit log (error type, failing atoms, repair instruction) unavailable when a strong generator silently fails. Under the adversarial `INJECT_TYPE_MISLEAD` ablation the weak gen + strong critic drops to 72.7%/28.6% on Gripper/Logistics, but in the *standard configuration* it achieves 100% on all domains.

13.4 Limitations

1. **Single-run evaluation.** Results are from single experimental runs without multiple seeds. PDDL generation has stochastic variation across API calls; reported percentages should be interpreted as estimates with $\pm$5–10% uncertainty. Future work should run each configuration at least three times and report mean $\pm$ standard deviation.
2. **No plan quality evaluation.** We measure only plan existence (solvability), not plan length or optimality. A shorter plan from the strong generator may be preferable even when both systems find a plan.
3. **STRIPS only.** All domains are STRIPS (no numerics, no durative actions). Extension to PDDL 2.1+ is future work.
4. **Single local model tested.** Only Qwen3.5 9B (Q4_K_M) was evaluated. Other models (Gemma4, Llama3.1, Qwen2.5 7B) available on the same hardware were not benchmarked. Extending evaluation to these models is future work.
5. **Problem files only; domain synthesis out of scope.** The system generates PDDL *problem* files given a trusted, human-authored domain file. Domain synthesis—generating `:action` schemas and predicate definitions from natural language—is a harder task requiring a different oracle, since correctness cannot be checked by a planner alone. Extending the critic to validate action precondition/effect semantics (e.g., via simulation or learned invariants) is an important direction for future work.
6. **Untyped STRIPS only; typed PDDL not supported.** The validator infers object-type membership via action-effect analysis, which applies only to untyped STRIPS domains where types are represented as ground atoms. Domains using PDDL `:typing` declarations require a different validation pass; the current type-predicate separation heuristic would misclassify typed `:init` atoms as missing state predicates, producing spurious repair instructions. Supporting typed PDDL is a direct and well-scoped extension.

14 Conclusion

We have presented a structured adversarial critic architecture for iterative PDDL repair, evaluated across four IPC benchmark domains with two cloud generator tiers, three repair strategies, and a local open-weights generator. Our key contributions are:

1. A five-class PDDL error taxonomy and critic prompt design that converts raw validator errors into targeted repair instructions.
2. A domain-agnostic PDDL validator that correctly handles untyped STRIPS domains by distinguishing type predicates from state predicates using action-effect analysis.
3. Empirical demonstration that critic quality, not generator size, is the decisive variable: weak gen + strong critic achieves **100%** (48/48) across all four domains with a strong critic—matching the strong generator + strong critic ceiling.
4. A controlled type-predicate ablation showing a +100 pp improvement from strong critic versus 0 pp from blind retry on Gripper standard configuration.
5. Validation of a fully-local generator deployment: **Qwen3.5 9B on a consumer RTX 3070 Ti (8 GB VRAM)** achieves 89.6% overall (43/48) with strong-critic repair—*exceeding* the strong cloud generator without repair (87.5%) at zero cloud generation cost.

The results suggest that future work should focus on critic specialization (domain-specific critics), critic fine-tuning on real PDDL repair traces, and extensions to richer PDDL dialects. The modular generator-critic architecture opens a concrete path to low-cost, privacy-preserving NL→PDDL translation that does not require frontier model access for generation.

Acknowledgements

Experiments were conducted on personal hardware (NVIDIA RTX 3070 Ti, 8 GB VRAM). Cloud LLM inference used Google AI Studio API credits. The author thanks the project supervisor for feedback on experimental design.

References

[1] McDermott, D., Ghallab, M., Howe, A., Knoblock, C., Ram, A., Veloso, M., Weld, D., & Wilkins, D. (1998). PDDL – The Planning Domain Definition Language. *AIPS-98 Planning Competition*.

[2] Guo, Y., et al. (2023). Can Large Language Models Generate PDDL Domains and Problems? A Study with GPT-4. *arXiv preprint arXiv:2305.06983*.

[3] Liu, B., et al. (2023). LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *arXiv preprint arXiv:2304.11477*.

[4] Valmeekam, K., et al. (2022). Large Language Models Still Can't Plan (A Benchmark for LLMs on Planning and Reasoning about Change). *NeurIPS 2022 Workshop on FMDM*.

[5] Silver, T., et al. (2022). PDDL Planning with Pretrained Large Language Models. *NeurIPS 2022 Workshop*.

[6] Xie, Y., et al. (2023). Translating Natural Language to Planning Goals with Large Language Models. *arXiv preprint arXiv:2302.09019*.

[7] Ahn, M., et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv preprint arXiv:2204.01691*.

[8] Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*.

[9] Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS 2023*.

[10] Huang, J., et al. (2023). Large Language Models Cannot Self-Correct Reasoning Yet. *arXiv preprint arXiv:2310.01848*.

[11] Chen, B., et al. (2022). CodeT: Code Generation with Generated Tests. *arXiv preprint arXiv:2207.10397*.

[12] Olausson, T., et al. (2023). Is Self-Repair a Silver Bullet for Code Generation? *ICLR 2024*.

[13] Alkhazraji, Y., et al. (2020). Pyperplan. `https://github.com/aibasel/pyperplan`.

[14] Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., & Kambhampati, S. (2023). PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. *NeurIPS 2023 Datasets and Benchmarks Track*.

[15] Kambhampati, S. (2024). Can Large Language Models Reason and Plan? *Annals of the New York Academy of Sciences*, 1534(1), 15–18. *arXiv preprint arXiv:2403.04121*.

[16] Zhou, A., et al. (2023). Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. *arXiv preprint arXiv:2310.04406*.

[17] Guan, L., Valmeekam, K., Sreedharan, S., & Kambhampati, S. (2023). Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning. *NeurIPS 2023*.

[18] Ni, A., et al. (2023). LEVER: Learning to Verify Language-to-Code Generation with Execution. *ICML 2023*.